

Development of a Python Framework to Manage Large Molecule Databases

Author : Baptiste ODET, *Tutor* : ABDUL RAHMAN ALLOUCHE

Institut Lumière Matière, UMR5306 CNRS, Université Claude Bernard Lyon 1

Source Code : <https://github.com/Castaneche/MolDataFW>

XTB2 Database : <https://doi.org/10.5281/zenodo.6722299>

ABSTRACT

Machine learning (ML) methods are extensively used to compute physical and chemical properties of large systems. Based on one or multiple neural networks, it is able to compute these properties faster than ab-initio methods, however they require a large amount of reliable data to learn. Here, we present the development of a code to extract and perform computation on the PubChemQC CHON database of 30 million molecules. This code can generate input files for Gaussian software using any ab-initio methods and can easily be extended to other chemical computation softwares. To validate our code, we successfully built a database using the most recent and most accurate semi-empirical method named XTB2. We found that clustering algorithms, such as KMeans, could be an interesting way to get a reduced database representative of the large one. That way, computations can be done on the reduced database and extended later after evaluation on a more large database. We finally trained a neural network with these data (500 molecules) and tested it on a larger dataset (27 000 molecules). It was able to compute molecular energy on unknown molecules with a relatively low mean absolute error (MAE) of about only 10 kcal/mol.

1. INTRODUCTION

When describing molecular potential energy, different strategies and methods are to consider. The most precise ones are those based on quantum mechanical approach. They basically try to solve the Schrödinger equation using various approximations and methods depending on the chosen algorithm, but they all suffer from a huge computational cost. It is very convenient on a small dataset as time computation stay reasonable, and it gives a very precise result. However, when dealing with huge dataset of molecules, ab-initio methods simply can't be processed. For instance, the time complexity of the density functional theory algorithm (DFT)^{1,2} or the more accurate (CCSD(T))³ method are respectively $O(N^3)$

and $O(N^7)$ for a system containing N electrons^{4,5}.

An alternative to ab-initio methods are semi-empirical methods, they get rid of some integral computations to reduce the time complexity at the cost of accuracy. According to the available time for this work, we couldn't use ab-initio methods, so we chose to use semi empirical XTB2 method which still have a reasonable accuracy.

In recent years, scientists explored different ways of computing molecular properties. Machine learning (ML) algorithms tend to be a good alternative to ab-initio methods, and they are already widely used to compute and predict molecular energy, forces or dipole⁶. It consists of a neural network trained on a dataset. Nevertheless, this technique requires a large

and reliable dataset to work on. By using an accurate dataset (depending on the problem we want to solve) neural networks find the solution faster than any rigorous computation method⁷. Main advantages of this technique are acceleration of development time, discovery of new molecules and molecular properties’ prediction.

We started from the PubChemQC PM6⁸ database, which is a large and diversified database. It contains 220 million molecules, optimized for the PM6 method⁹. We focus on 30 million organic molecules constituted of hydrogen, carbon, nitrogen and oxygen atoms. The goal is to provide python scripts to process this molecule database easily. As there are not so many available code base online to achieve this, we developed the code and made it open source to allow researchers to reproduce or use it. Because PM6 method isn’t precise enough, we developed a framework to build a database with any other more precise quantum mechanic method. Computing of energy and forces with DFT is very consuming time and can not be build during the 6 weeks of my internship. So we evaluate our code using XTB2, very fast and more precise than PM6. Then, We successfully trained a ML algorithm on the reduced database and tested it on a larger set of data. Other goal of our work is the reduction of training time by reduction of database.

Firstly, we will describe the code used for this work. Then we will discuss techniques to effectively reduced the database. Finally, we will show the results of a machine learning algorithm to compute molecular energy.

2. CODE STRUCTURE

This section describes all developed code in this work. In the first step, we will describe the code we have developed to convert the JSON files of PubChem database, in the Hierarchical Data Format (HDF). In second step we will describe our developed code to generate input files for computational softwares and then to build a new database with more precise properties (as energies, forces, ...). Finally, we explore some techniques to reduce and analyze the database. Each part explained here is referenced in the diagram (Fig. 2).

2.1. Extracting and Processing Raw Data. We provide a versatile way of extracting and processing this database. This is done in three steps : the extraction and conversion of data, exportation of

data to a readable format for Gaussian software and extraction of output file. Each database we produce is one or multiple HDF files containing data for each molecule. We chose this format mainly because it’s a compressed and hierarchical file.

2.1.1. GeomData. This script extracts data from the PubChem database. For each molecule inside the JSON files, we only extract useful data : identifiers, molecular formula, PM6 energy, atoms, atom coordinates, charge and spin multiplicity. We now have a starting database stored as HDF.

2.1.2. ComData. We then export data to a .com file which is readable by Gaussian software. Gaussian perform computation of structure, energy, forces, dipole, quadrupole, depending on the method we use. For this work, and for time reason we will discuss later, we chose to perform XTB2 method, but any computation could be achieved. Note here that our code generates input files for Gaussian but it is easily modifiable to generate input files for other quantum chemistry software such as MOPAC, and Orca for example.

2.1.3. LogData. This script reads the output .log file provided by Gaussian software and extract all the computed data (described before). We then merge these data with the one extracted using *GeomData* to obtain a XTB2 database.

2.2. Code for Reducing DataBase. We built scripts to reduce the database to a smaller one using two methods : random and clustering. *ReduceRandom* script select randomly a subset of the data. *wACSF* and *Clustering* scripts determine atomic environment function and clusters data using a clustering algorithm. They are both described in the Results part later. Metrics is a Jupyter notebook containing multiple python scripts that can output histograms, tables or graphs to analyze the HDF data files. This is necessary for this work to characterize databases and to determine which reduction method to use.

2.3. Extras. We provide extra scripts for testing or do specific tasks. First, we have the *EnergyCorrection* script that performs a subtraction on energy to retrieve the dissociation energy. It simply subtracts each atom’s energy to the total energy. Next, we have a *ConvertData* script that provides a way to convert from HDF file to N2P2 file. This is needed because the machine learning code we use only takes a N2P2 file as input. Finally, *Anomalies* script filters the database by excluding outliers molecules with diverging energy, forces, dipoles...

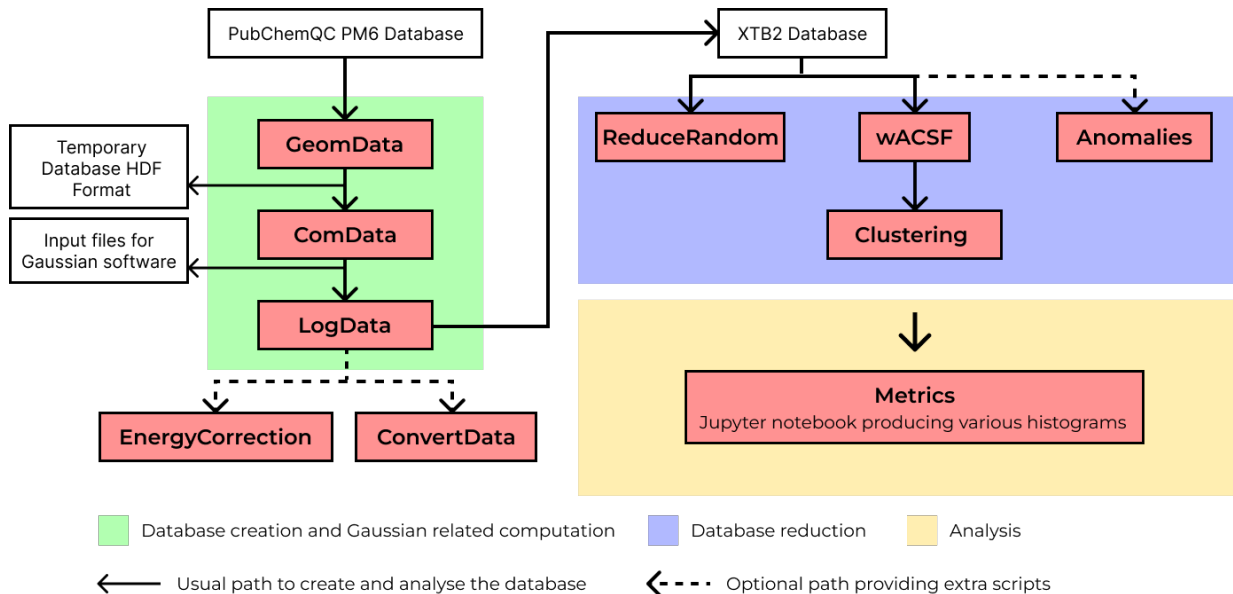


Figure 2: Code structure schematized. White squares correspond to databases and red ones to python scripts. Extra scripts, performing some adjustments or conversions, are linked using dashed arrows. First (green) block is the pipeline to extract data from the PubChem database and perform calculation using Gaussian software. Second (blue) block is the pipeline to reduce the database, providing two ways of doing it (see Database Reduction section). It is linked to the XTb2 Database because we chose to generate such a database, but it should work with any as long as it remains in the same format. Last (yellow) block is related to analysis and provide a notebook where some figures shown here come from.

3. RESULTS

3.1. XTb2 Database. The PM6 method is fast but not very precise. To get the most out of the ML algorithm, we chose to build a new database using a more precise method. Density functional theory (DFT) would be a very precise solution, but with 30 million molecules it will take years to compute. To be able to do some analysis and testing, we trade off DFT for XTb2 semi-empirical method. This method is less precise but much faster. We used Gaussian software to compute properties of the whole database. The analysis and ML will be done on this XTb2 database.

3.2. Database Reduction. The database contains approximately 30 million molecules with, for each one of them, coordinates, energy, forces, dipole, quadrupole... In order to process the data (to feed a ML algorithm, for example), we need a way to generate a reduced database representative of the large one.

3.2.1. Randomly. The naïve approach is by selecting a random sample from the database in two different ways : by molecule or by atom. Selecting by molecule is straight forward, we select a percentage of molecules constituting the database. We call this method 'random (mol)'. Selecting by atom is not so

complicated either, we unpack molecules into atoms, then we perform a random selection over atoms and retrieve the molecules they are bound to. We call this one 'random (atom)'. Results of this approach are discussed later.

3.2.2. Clustering. The clustering technique might be a way to efficiently reduce the database. The concept is simple, the algorithm packs similar atom into clusters, then we select a portion of each cluster to create a reduced database representative of the large one. The idea is to cluster atoms according to their atomic environment. This way, we are assured to keep every type of atom binding and thus to keep a diversity of molecule. We need a function that can describe an atomic environment, a common way to achieve this is by using a descriptor (that we call G function) which is a radial symmetry function^{10,11}, defined as follows :

$$G_i^{rad} = \sum_{j \neq i}^N Z_j e^{-\eta(r_{ij}-\mu)} f_c(r_{ij}) \quad (1)$$

Where N is the number of atoms in the molecule and r_{ij} the distance between atom i and j. η and μ modulates the width and position of the Gaussian function. The cutoff function f_c restricts the region of interaction by setting up a cut-off radius r_c and is defined

as follows :

$$f_c(r_{ij}) = \begin{cases} \frac{1}{2} [\cos(\frac{\pi r_{ij}}{r_c}) + 1], & r_{ij} \leq r_c \\ 0, & r_{ij} > r_c \end{cases} \quad (2)$$

This descriptor is very convenient because we only need atom coordinates to be able to describe an atom environment, no need for extra calculation (energy, forces...). To have a large description of the atom environment, we compute ten functions for each atom with varying parameters (see Fig. 3).

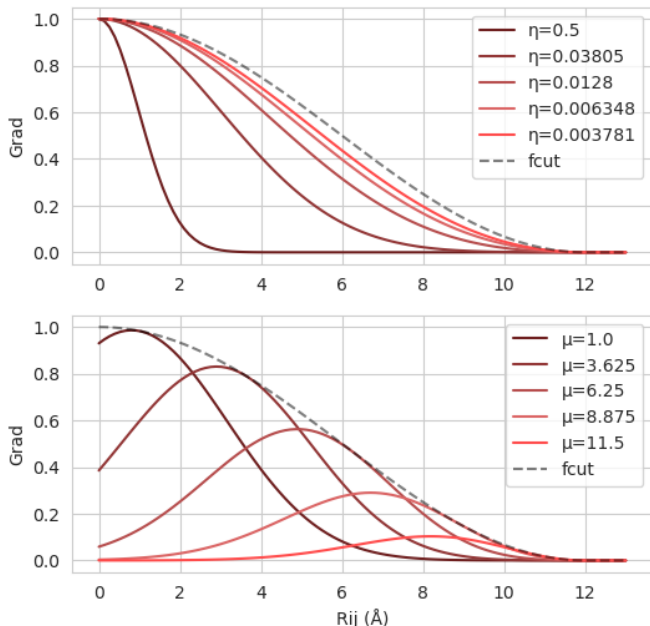


Figure 3: Radial Gaussian functions generated for this work (total of ten functions). The upper panel shows functions with a constant $\mu = 0$ when the lower panel shows functions with constant $\eta = 0.07256$. The Gaussian functions are multiply by the cut-off function (dashed curve). Cut-off radius is constant and set to 12 Bohr.

The algorithm used for clustering is called KMeans. It takes as input the number of clusters we want and precomputed G functions for each atom. For explanatory purposes, let's take a two-dimensional dataset (x,y). The first step of the algorithm is to define centroids. Centroid is a center of a cluster and initially set to a random point. The next step assign each data point to the closest cluster by calculating euclidean distance. Finally, centroids are re-initialized by calculating the average of data points inside the cluster. These last two steps are repeated until we get optimal centroids. This way, each data is packed with the similar ones. In this work, the KMeans algorithm works in ten dimensions instead of two because we computed ten radial functions, but the concept remains the same.

The KMeans algorithms clusters data by minimizing a criterion called inertia or within-cluster sum-of-squares. By computing inertia cost for several number of cluster (k) (Fig 4), we get acceptable inertia for a number of clusters above 20. Computation time of the KMeans algorithm is depending on k, so we didn't go further than 50 and we chose two values for k : 20 and 50. We call these two new methods of reducing the database '20clusters' and '50clusters'.

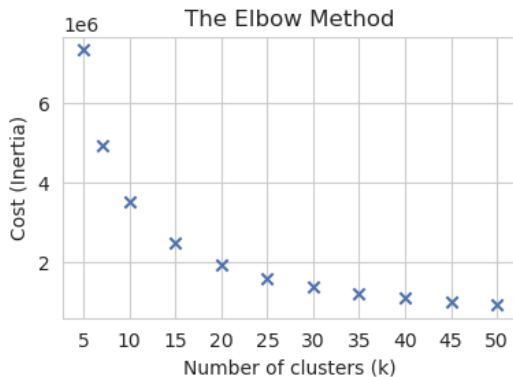


Figure 4: Inertia cost against number of clusters

3.2.3. Analyze. For the purpose of testing and because we don't have the time to compute for the 30 million molecules, we take 11190 molecules (we call this the original database) and perform four types of reduction : random by molecule, random by atom, clustering with k=20 and clustering with k=50. We ensure to reduce the database to 500 molecules (95.53% reduction) for all types of reduction.

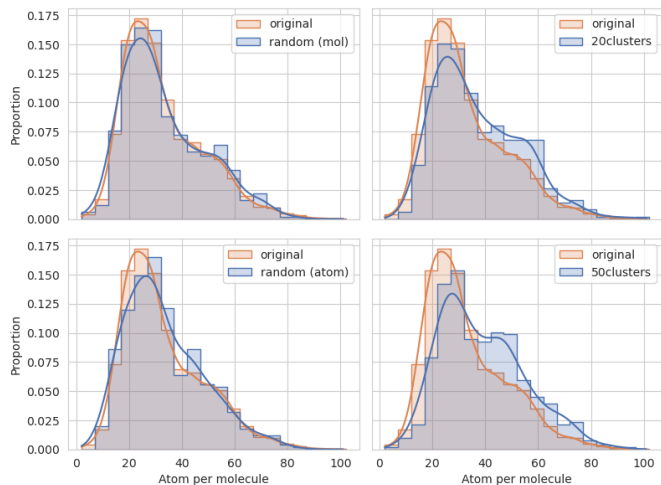


Figure 5: Number of atoms per molecule comparison between the original database and reduced one. Both histograms and density functions are represented.

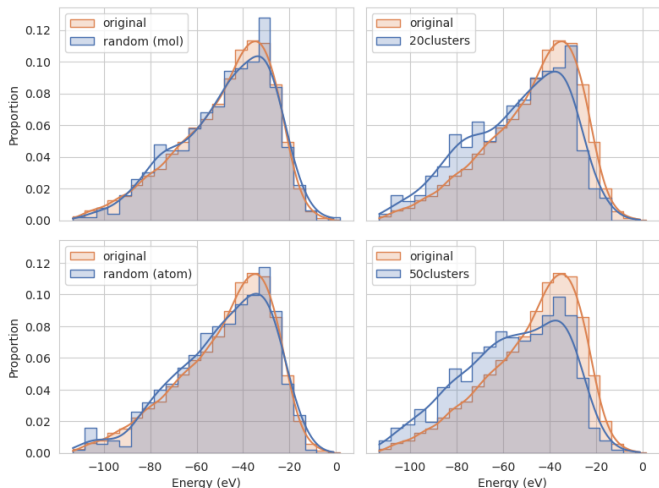


Figure 6: Energy (XTB2) comparison between the original database and reduced ones. Both histograms and density functions are represented.

Selecting a random sample by molecule doesn't really change the distributions, as we can see on Fig. 5 and 6. Distribution of random (atom) method introduce a slightly different distribution of atom per molecule. On the other hand, reduction using a clustering technique change the distribution of atom per molecule as well as the distribution of energy of our original database. It seems to provide more variety of molecule (in proportion) than in the original database. Furthermore, the proportion of atom type in each reduced database is pretty similar to the proportion in the original database (Table 1). We can safely say that reducing database using clustering bring more diversity of molecules than the random method and does not alter the overall proportion of atoms.

	H	C	N	O	Total
original	188954 51.55%	140147 38.23%	13066 3.56%	24404 6.66%	366571 100%
random (mol)	8624 52.22%	6256 37.88%	570 3.45%	1066 6.45%	16516 100%
random (atom)	8571 51.42%	6333 37.99%	685 4.11%	1080 6.48%	16669 100%
20clusters	9453 51.34%	7032 38.19%	655 3.56%	1274 6.92%	18414 100%
50clusters	8874 51.66%	6498 37.83%	588 3.42%	1219 7.1%	17179 100%

Table 1: Number of atoms and proportion relative to the total number of atom for each reduction method.

3.3. Test. We trained a neural network on XTB2 reduced database, which goal was to recognize patterns to be able to predict molecular energy. The train set was about 400 molecules and validation set a 100. We then picked about 27 000 molecules from

the XTB2 databases and tried to predict energy. We then compare XTB2 energy and the one computed by the ML algorithm (Fig. 7). Turns out that 'random (mol)' method failed to compute the correct energies. Selection by atom and clustering techniques predicted energy correctly with a linear coefficient close to 1. Moreover, the computation ended with a low mean absolute error (MAE) of about 10 kcal/mol which is a relatively good precision. It is too early to say that clustering techniques are efficient because we did only one test according to the remaining time of this internship. But it might have a positive impact and should be chosen over the naïve random selection by molecule. In any case, random (mol) method is to avoid.

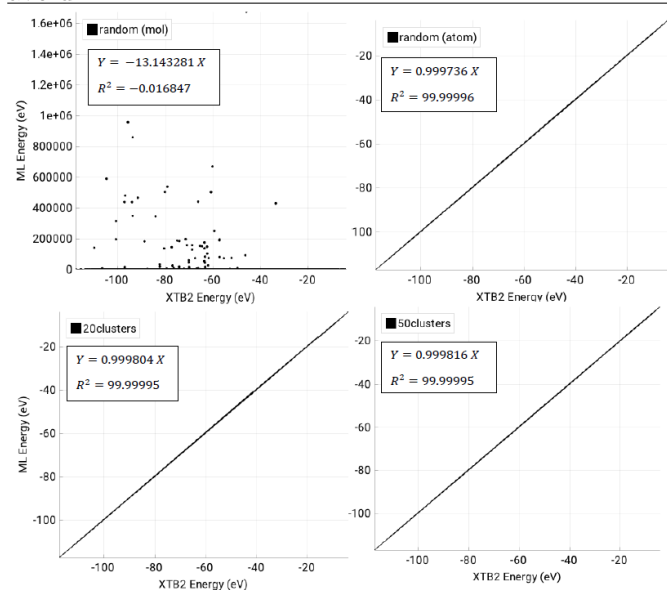


Figure 7: Energy computed using the machine learning code (MLEnergy) against energy computed using XTB2 method. Linear regression output is shown over graphs.

4. CONCLUSION

During this internship, we developed a code for managing a large molecule database. Every step, from extraction to exportation, is thought to be the most versatile to be able to produce multiple outputs. We built a full database of molecules using XTB2 method. We trained a ML algorithm on this database to compute molecular energy faster. We also explored techniques to reduce the database and be able to perform fewer calculations.

Code is fully functional, open source and published with a non-restrictive license to allow future researchers to reproduce or extend the sources. It is optimized to work with large amount of data and

can be parallelized on multiple CPU cores to perform faster calculations. The tests done on the database reduction part show that random molecule selection is to avoid, but random atom selection and clustering techniques are efficient ways of reducing the database. Training the neural network on reduced database worked, and we were able to compute molecular energy for many molecules much more rapidly than ab-initio methods.

Due to the time available, we only did one test for the machine learning part, thus we can't be really confident. Moreover, all the tests we did in results part, are done on a subset (27 000 molecules) of the large database of 30 million molecules. Ideally, we should use the whole database to perform reduction on.

Future works could explore the machine learning part deeper. This includes more testing to be extremely confident in the techniques used here. Other tracks can be explored, for instance, we could also perform clustering with a higher number of G functions per atom (10 are used here), or we could select a different number of clusters k (20 and 50 used here). Also, a more precise database could be used and generated. For instance, the time-consuming DFT method could be used to compute properties for a reduced set of molecules, generated using database reduction methods, and extended to a large database using machine learning techniques.

REFERENCES

- [1] P. Hohenberg and W. Kohn. Inhomogeneous electron gas. *Phys. Rev.*, 136:B864–B871, Nov 1964. doi: 10.1103/PhysRev.136.B864. URL <https://link.aps.org/doi/10.1103/PhysRev.136.B864>.
- [2] W. Kohn and L. J. Sham. Self-consistent equations including exchange and correlation effects. *Phys. Rev.*, 140:A1133–A1138, Nov 1965. doi: 10.1103/PhysRev.140.A1133. URL <https://link.aps.org/doi/10.1103/PhysRev.140.A1133>.
- [3] Claudia Hampel, Kirk A. Peterson, and Hans-Joachim Werner. A comparison of the efficiency and accuracy of the quadratic configuration interaction (qcisd), coupled cluster (ccsd), and brueckner coupled cluster (bccd) methods. *Chemical Physics Letters*, 190(1):1–12, 1992. ISSN 0009-2614. doi: [https://doi.org/10.1016/0009-2614\(92\)86093-W](https://doi.org/10.1016/0009-2614(92)86093-W). URL <https://www.sciencedirect.com/science/article/pii/000926149286093W>.
- [4] Mikael Johansson, Ville Kaila, and Dage Sundholm. Ab initio, density functional theory, and semiempirical calculations. *Methods in molecular biology (Clifton, N.J.)*, 924:3–27, 08 2013. doi: 10.1007/978-1-62703-017-5_1.
- [5] Hatice Gokcan and Olexandr Isayev. Learning molecular potentials with neural networks. *WIREs Computational Molecular Science*, 12(2):e1564, 2022. doi: <https://doi.org/10.1002/wcms.1564>. URL <https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/wcms.1564>.
- [6] Oliver T. Unke and Markus Meuwly. Physnet: A neural network for predicting energies, forces, dipole moments, and partial charges. *Journal of Chemical Theory and Computation*, 15(6):3678–3693, 2019. doi: 10.1021/acs.jctc.9b00181. URL <https://doi.org/10.1021/acs.jctc.9b00181>. PMID: 31042390.
- [7] Cong Huy Pham, Rebecca K. Lindsey, Laurence E. Fried, and Nir Goldman. High-accuracy semiempirical quantum models based on a minimal training set. *The Journal of Physical Chemistry Letters*, 13(13):2934–2942, 2022. doi: 10.1021/acs.jpclett.2c00453. URL <https://doi.org/10.1021/acs.jpclett.2c00453>. PMID: 35343698.
- [8] Maho Nakata, Tomomi Shimazaki, Masatomo Hashimoto, and Toshiyuki Maeda. Pubchemqc pm6: Data sets of 221 million molecules with optimized molecular geometries and electronic properties. *Journal of Chemical Information and Modeling*, 60(12):5891–5899, 2020. doi: 10.1021/acs.jcim.0c00740. URL <https://doi.org/10.1021/acs.jcim.0c00740>. PMID: 33104339.
- [9] James J. P. Stewart. Optimization of parameters for semiempirical methods v: Modification of nddo approximations and application to 70 elements. *Journal of Molecular Modeling*, 13(12):1173–1213, Dec 2007. ISSN 0948-5023. doi: 10.1007/s00894-007-0233-4. URL <https://doi.org/10.1007/s00894-007-0233-4>.
- [10] M. Gastegger, L. Schwiedrzik, M. Bittermann, F. Berzsényi, and P. Marquetand. wacs—weighted atom-centered symmetry functions as descriptors in machine learning potentials. *The Journal of Chemical Physics*, 148(24):241709, 2018. doi: 10.1063/1.5019667. URL <https://doi.org/10.1063/1.5019667>.
- [11] Giulio Imbalzano, Andrea Anelli, Daniele Giofré, Sinja Klees, Jörg Behler, and Michele Ceriotti. Automatic selection of atomic fingerprints and reference configurations for machine-learning potentials. *The Journal of Chemical Physics*, 148(24):241730, 2018. doi: 10.1063/1.5024611. URL <https://doi.org/10.1063/1.5024611>.